THE UNIVERSITY OF AZAD JAMMU AND KASHMIR, MUZAFFARABAD



COURSE TITLE	Digital Imaging Processing
COURSE CODE	CS-3112
LAB TITLE	Image Enhancement & Intensity Transformation
STUDENT NAME	MAHDI ALI NAHVI
STUDENT ROLL NUMBER	2024-SE-16
INSTRUCTOR NAME	Engr. Fahad Nisar
SUBMISSION DATE	November 04, 2026
LAB NUMBER	03

Objective

To calculate, implement and compare grayscale intensity transformations, histogram enhancement and thresholding, while preventing overflow and interpreting the effect of each operation. All 11 operations were implemented in Python (NumPy, OpenCV, Matplotlib) and each one is supported by a hand-calculated dry run that was verified in Python.

0. Image loading and numeric safety

The selected photograph (a monarch butterfly on lavender) is a colour JPG, so it was converted once with cv2.cvtColor to grayscale, saved as Butterfly_gray.png and re-loaded with cv2.IMREAD_GRAYSCALE. A separate copy (original) is kept and the working array is write-protected so that no operation can change it.

```
color = cv2.imread("Butterfly.jpg")
print("Colour image shape:", color.shape)

gray_conv = cv2.cvtColor(color, cv2.COLOR_BGR2GRAY)
cv2.imwrite("Butterfly_gray.png", gray_conv)

img = cv2.imread("Butterfly_gray.png", cv2.IMREAD_GRAYSCALE)
assert img is not None, "image failed to load"
assert img.ndim == 2 and img.dtype == np.uint8, "not an 8-bit grayscale image"
assert img.shape[0] >= 300 and img.shape[1] >= 300, "image smaller than 300x300"

original = img.copy()          # untouched reference copy
img.flags.writeable = False    # protects img from accidental in-place changes

print("shape :", img.shape)
print("dtype :", img.dtype)
print("min   :", img.min())
print("max   :", img.max())

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
ax[0].imshow(cv2.cvtColor(color, cv2.COLOR_BGR2RGB)); ax[0].set_title("Original colour image"); ax[0].axis("off")
show(ax[1], img, "Grayscale image used in this lab")
plt.tight_layout(); save_fig("00_load"); plt.show()
```

Property	Value
shape	(768, 1408)
dtype	uint8
minimum	0
maximum	255
mean / median / std	118.16 / 116 / 47.58

Table 0 – Image properties (output of the notebook)



Figure 0 – Colour input image and the grayscale image used in the lab.

Numeric safety: uint8 arithmetic wraps around (e.g. 250 + 40 = 34). In the notebook, plain uint8 addition of +40 wrapped 17,554 pixels, which would have turned bright pixels dark. Therefore every calculation uses int16 / float types, the result is clipped to 0–255 with np.clip and only then converted to uint8 (astype truncates the fraction). All images are displayed with vmin = 0 and vmax = 255 so that Matplotlib does not auto-stretch the contrast.

1. Brightness adjustment (β = +40 and −40)

```
# +40 using nested loops (slow but explicit) and comparison with NumPy
h, w = img.shape
loop_bright = np.zeros_like(img)
for i in range(h):
    for j in range(w):
        v = int(img[i, j]) + 40
        if v > 255:
            v = 255
        loop_bright[i, j] = v

print("Nested-loop result identical to NumPy result:", np.array_equal(loop_bright, bright))
```

```
Nested-loop result identical to NumPy result: True
```



Figure 1 – Original, brighter (+40) and darker (−40) images.

Python output: mean 118.16 → 157.96 (+40) and 79.31 (−40); 18,559 pixels saturate at 255 in the +40 image and 69,180 pixels become 0 in the −40 image. The nested-loop implementation is identical to the NumPy result (np.array_equal = True).

Dry run 1 – pixels [30, 80, 128, 180, 240]

r	β	raw r+β	clipped	final uint8
30	+40	70	70	70
30	-40	-10	0	0
80	+40	120	120	120
80	-40	40	40	40
128	+40	168	168	168
128	-40	88	88	88
180	+40	220	220	220
180	-40	140	140	140
240	+40	280	255	255
240	-40	200	200	200

Table 1 – Brightness dry run

Lost detail: with +40, 240 → 280 is clipped to 255, so every input from 215 to 255 collapses onto 255 (highlights lost). With −40, 30 → −10 is clipped to 0, so every input from 0 to 40 collapses onto 0

(shadows lost). Clipped information cannot be restored by applying the opposite shift. Verified in Python: NumPy +40 = [70 120 168 220 255], −40 = [0 40 88 140 200].

2. Linear contrast from zero: s = αr + β (α = 1.5, β = 0)

```
lin15 = to_u8(img.astype(np.float32) * 1.5 + 0)

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
show(ax[0], img,   "Original")
show(ax[1], lin15, "s = 1.5 r  (α = 1.5, β = 0)")
plt.tight_layout(); save_fig("02_linear_contrast"); plt.show()

print("Mean original / 1.5r        :", round(img.mean(),2), round(lin15.mean(),2))
print("Pixels = 255 original / 1.5r:", int(np.sum(img==255)), int(np.sum(lin15==255)))
print("Share saturated at 255 (1.5r): %.2f %%" % (100*np.mean(lin15==255)))
print("All inputs >= 170 saturate  :", bool(np.all(lin15[img >= 170] == 255)))
```



Figure 2 – Original versus s = 1.5 r.

The mean rises from 118.16 to 172.00, so the image is much brighter as well as higher in contrast. The count of pixels equal to 255 grows from 10 to 184,219 (17.04 % of the image); every input ≥ 170 saturates, so the bright areas of the wings and the pale flowers lose their texture.

Dry run 2 – pixels [30, 80, 128, 180, 240]

r	1.5 r (raw)	clipped	truncated uint8	saturated?
30	45	45	45	no
80	120	120	120	no
128	192	192	192	no
180	270	255	255	YES
240	360	255	255	YES

Table 2 – Linear contrast dry run

Saturated pixels: 180 (270) and 240 (360). Both become 255 and can no longer be told apart. NumPy check: [45 120 192 255 255].

3. Contrast centred at 128: s = (r − 128)α + 128

```
def centered(im, a):
    return to_u8((im.astype(np.float32) - 128) * a + 128)

c15 = centered(img, 1.5)
c05 = centered(img, 0.5)

fig, ax = plt.subplots(2, 2, figsize=(12, 7))
show(ax[0,0], img,   "Original")
show(ax[0,1], lin15, "Linear from zero: 1.5 r")
show(ax[1,0], c15,   "Centred at 128, α = 1.5")
show(ax[1,1], c05,   "Centred at 128, α = 0.5")
plt.tight_layout(); save_fig("03_centered_contrast"); plt.show()

stats = pd.DataFrame({
    "mean":   [img.mean(), lin15.mean(), c15.mean(), c05.mean()],
    "std":    [img.std(),  lin15.std(),  c15.std(),  c05.std()],
    "% at 0": [100*np.mean(a==0)   for a in (img, lin15, c15, c05)],
    "% at 255":[100*np.mean(a==255) for a in (img, lin15, c15, c05)],
}, index=["original", "1.5 r", "centred 1.5", "centred 0.5"]).round(2)
display(stats)
DRY["stats3"] = stats.reset_index().values.tolist()
```

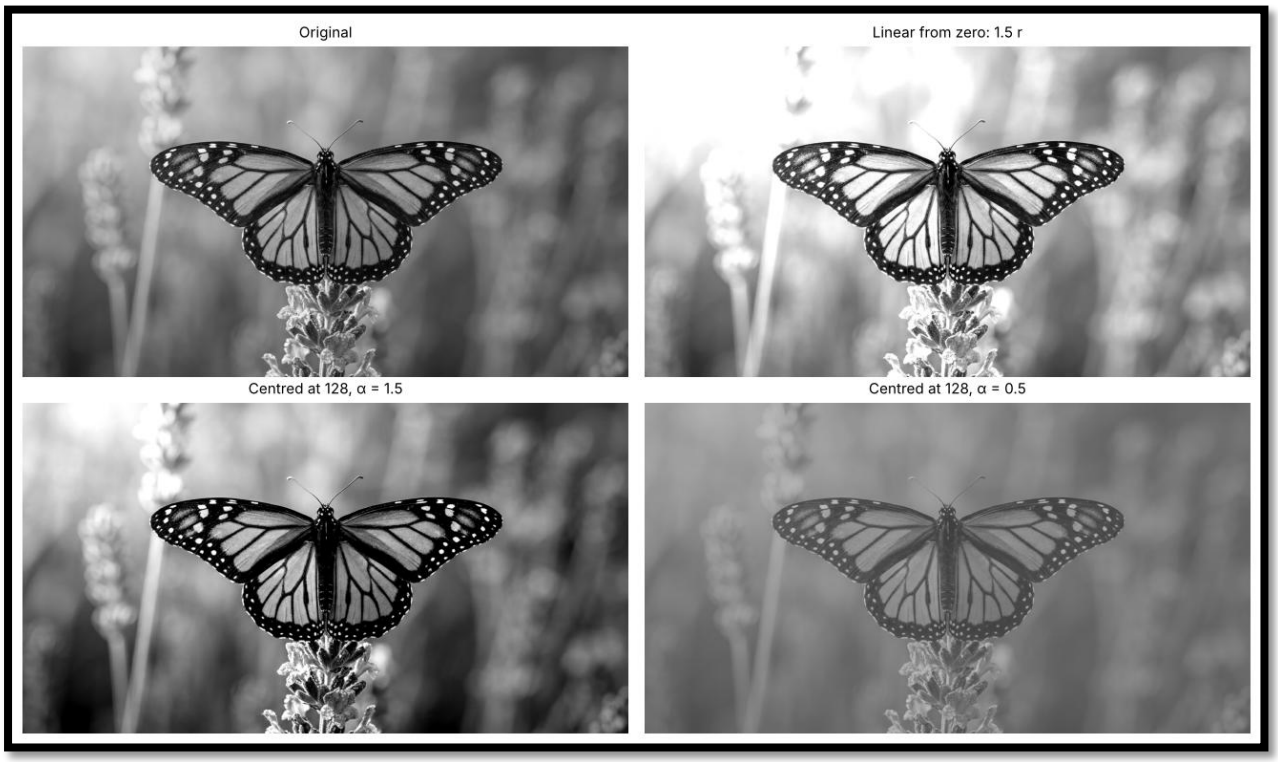


Figure 3 – Original, 1.5 r, centred α = 1.5 and centred α = 0.5.

Image	mean	std	% at 0	% at 255
original	118.16	47.58	0.04	0
1.5 r	172	63.57	0.04	17.04
centred 1.5	114.63	66.72	6.89	1.94
centred 0.5	122.83	23.79	0	0

Table 3 – Statistics of the four images

Comparison: s = 1.5 r raised the mean from 118 to 172 (a brightness change) with 17 % saturated pixels. Centred contrast with α = 1.5 keeps the mean near 115 (almost unchanged) while the standard deviation rises from 47.6 to 66.7; it clips shadows (6.9 % at 0) and a few highlights (1.9 % at 255) symmetrically. α = 0.5 halves the spread (std 23.8) and gives a flat, low-contrast image with no clipping.

Dry run 3 – pixels [30, 80, 128, 180, 240]

r	α	r−128	(r−128)α	+128 raw	clipped	truncated
30	1.5	-98	-147	-19	0	0
30	0.5	-98	-49	79	79	79
80	1.5	-48	-72	56	56	56
80	0.5	-48	-24	104	104	104
128	1.5	0	0	128	128	128
128	0.5	0	0	128	128	128
180	1.5	52	78	206	206	206
180	0.5	52	26	154	154	154
240	1.5	112	168	296	255	255
240	0.5	112	56	184	184	184

Table 4 – Centred contrast dry run

At r = 128 the term (r − 128) is zero, so the output is 128 for any α: the centre of the range is a fixed point. NumPy check: α = 1.5 → [0 56 128 206 255]; α = 0.5 → [79 104 128 154 184].

4. Image negative: s = 255 − r

```
neg = 255 - img                    # uint8: 255 - r never goes below 0, so no overflow
neg2 = 255 - neg

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
show(ax[0], img, "Original")
show(ax[1], neg, "Negative (255 - r)")
plt.tight_layout(); save_fig("04_negative"); plt.show()

print("Negative of negative equals original:", np.array_equal(neg2, img))
```



Figure 4 – Original and negative image.

The negative reverses the intensity scale; applying it twice returns the original exactly (verified: True). Useful situation: in medical images such as X-rays or mammograms, small bright structures in dark regions are often easier to inspect as dark detail on a bright background.

Dry run 4 – pixels [30, 80, 128, 180, 240]

r	255 – r	negative again	equals original?
30	225	30	yes
80	175	80	yes
128	127	128	yes
180	75	180	yes
240	15	240	yes

Table 5 – Negative dry run

5. Gamma correction: s = 255 (r/255)^γ (γ = 0.5 and 2.0)

```
def gamma_tf(im, g):
    return to_u8(255.0 * (im.astype(np.float64) / 255.0) ** g)

g05 = gamma_tf(img, 0.5)
g20 = gamma_tf(img, 2.0)

fig, ax = plt.subplots(1, 3, figsize=(15, 4))
show(ax[0], img, "Original")
show(ax[1], g05, "Gamma 0.5 (brighter)")
show(ax[2], g20, "Gamma 2.0 (darker)")
plt.tight_layout(); save_fig("05_gamma"); plt.show()

# effect on dark regions (pixels < 64 in the original)
dark_mask = img < 64
print("Mean of dark-region pixels  original:", round(img[dark_mask].mean(),2),
      "| γ=0.5:", round(g05[dark_mask].mean(),2), "| γ=2.0:", round(g20[dark_mask].mean(),2))

# 0 and 255 stay fixed
for g in (0.5, 2.0):
    print(f"γ={g}: 0 ->", gamma_tf(np.array([0]), g)[0], "  255 ->", gamma_tf(np.array([255]), g)[0])
```



Figure 5 – Original, γ = 0.5 and γ = 2.0.

Gamma convention: the output is 255·(r/255)^γ, so γ < 1 brightens (lifts dark regions) and γ > 1 darkens. For pixels below 64 in the original (mean 35.04) the mean becomes 90.10 with γ = 0.5 (dark detail becomes visible) and 5.60 with γ = 2.0 (shadows are crushed). Confirmed in Python that 0 → 0 and 255 → 255 for both γ.

Dry run 5 – pixels [0, 64, 128, 192, 255]

r	γ	r/255	(r/255)^γ	255·(…)	truncated
0	0.5	0	0	0	0
0	2	0	0	0	0
64	0.5	0.251	0.501	127.7498	127
64	2	0.251	0.063	16.0627	16
128	0.5	0.502	0.7085	180.6654	180
128	2	0.502	0.252	64.251	64
192	0.5	0.7529	0.8677	221.2691	221
192	2	0.7529	0.5669	144.5647	144
255	0.5	1	1	255	255
255	2	1	1	255	255

Table 6 – Gamma dry run (full precision; truncation only at the end)

Difference from the rounded slide example: if the slide rounds r/255 (64/255 = 0.2510 → 0.25) before taking the square root, it gets 255 × 0.5 = 127.5 → 128. With full precision, 255 × 0.500979 = 127.75, which truncates to 127. The one-level difference comes from early rounding and from rounding versus truncating. NumPy check: γ = 0.5 → [0 127 180 221 255]; γ = 2.0 → [0 16 64 144 255].

6. Grayscale histogram

```
plt.figure(figsize=(9, 4))
plt.hist(img.ravel(), bins=256, range=(0, 256), color="gray")
plt.title("Histogram of the original grayscale image (256 bins)")
plt.xlabel("Intensity (0-255)"); plt.ylabel("Number of pixels")
plt.xlim(0, 256); plt.grid(alpha=0.3)
save_fig("06_histogram"); plt.show()

hist = np.bincount(img.ravel(), minlength=256)
print("Most frequent intensity:", hist.argmax(), "with", hist.max(), "pixels")
print("Mean %.2f | median %.1f | std %.2f" % (img.mean(), np.median(img), img.std()))
print("Share < 64: %.1f %% | 64-191: %.1f %% | >= 192: %.1f %%" %
      (100*np.mean(img<64), 100*np.mean((img>=64)&(img<192)), 100*np.mean(img>=192)))
```

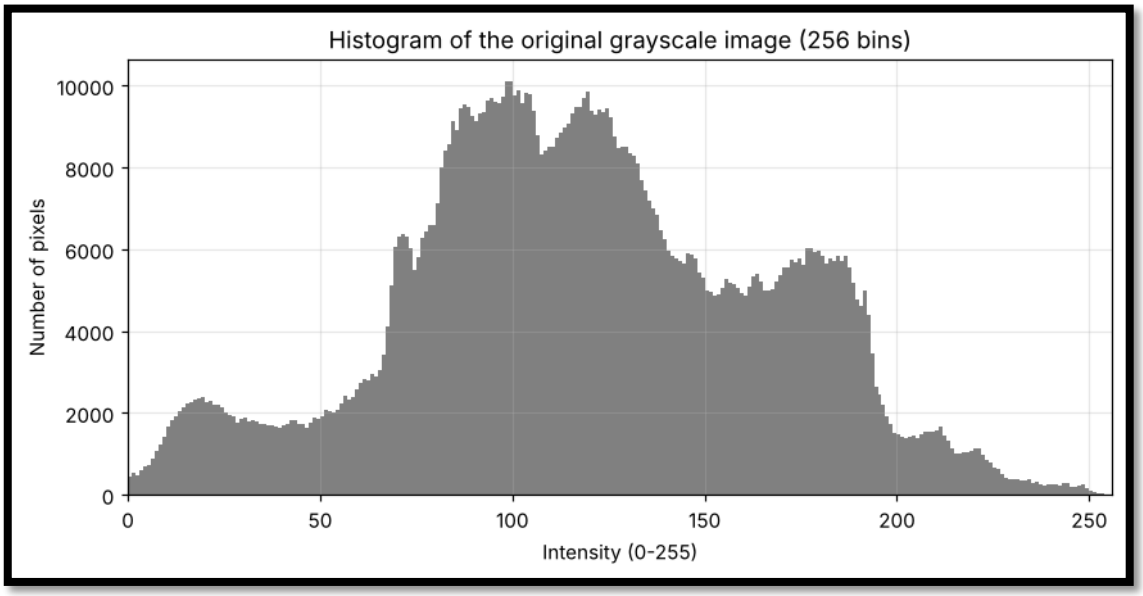


Figure 6 – 256-bin histogram of the original image.

Interpretation: the distribution is concentrated in the mid-tones – about 83 % of the pixels lie between 64 and 191, with a main peak at intensity 98 (10,122 pixels) and a mean of 118. Only about 11 % of pixels are dark (< 64) and about 6 % are bright (≥ 192), so the image has moderate contrast and does not use the full dynamic range well.

Dry run 6 – A = [[0, 50, 50, 100], [100, 150, 200, 255]]

intensity	frequency	probability
0	1	0.125
50	2	0.25
100	2	0.25
150	1	0.125
200	1	0.125
255	1	0.125

Table 7 – Frequencies and probabilities (total pixels = 8, sum of probabilities = 1)

A histogram counts how many pixels have each intensity but not where they are. Rearranging, shuffling or rotating the same pixels gives an identical histogram, so all spatial information is lost.

7. Global histogram equalization

```python
eq = cv2.equalizeHist(img)

fig, ax = plt.subplots(2, 2, figsize=(13, 8))
show(ax[0,0], img, "Original")
show(ax[0,1], eq,  "Globally equalized")
ax[1,0].hist(img.ravel(), bins=256, range=(0,256), color="gray"); ax[1,0].set_title("Original histogram")
ax[1,1].hist(eq.ravel(),  bins=256, range=(0,256), color="gray"); ax[1,1].set_title("Equalized histogram")
for a in ax[1]: a.set_xlim(0,256); a.set_xlabel("Intensity"); a.set_ylabel("Pixels")
plt.tight_layout(); save_fig("07_equalization"); plt.show()

# simple noise / detail proxy: std of the Laplacian (high-frequency energy)
lap = lambda im: cv2.Laplacian(im, cv2.CV_64F).std()
print("Std of intensities  original / equalized:", round(img.std(),2), round(eq.std(),2))
print("Laplacian std (detail+noise) original / equalized:", round(lap(img),2), round(lap(eq),2))
```

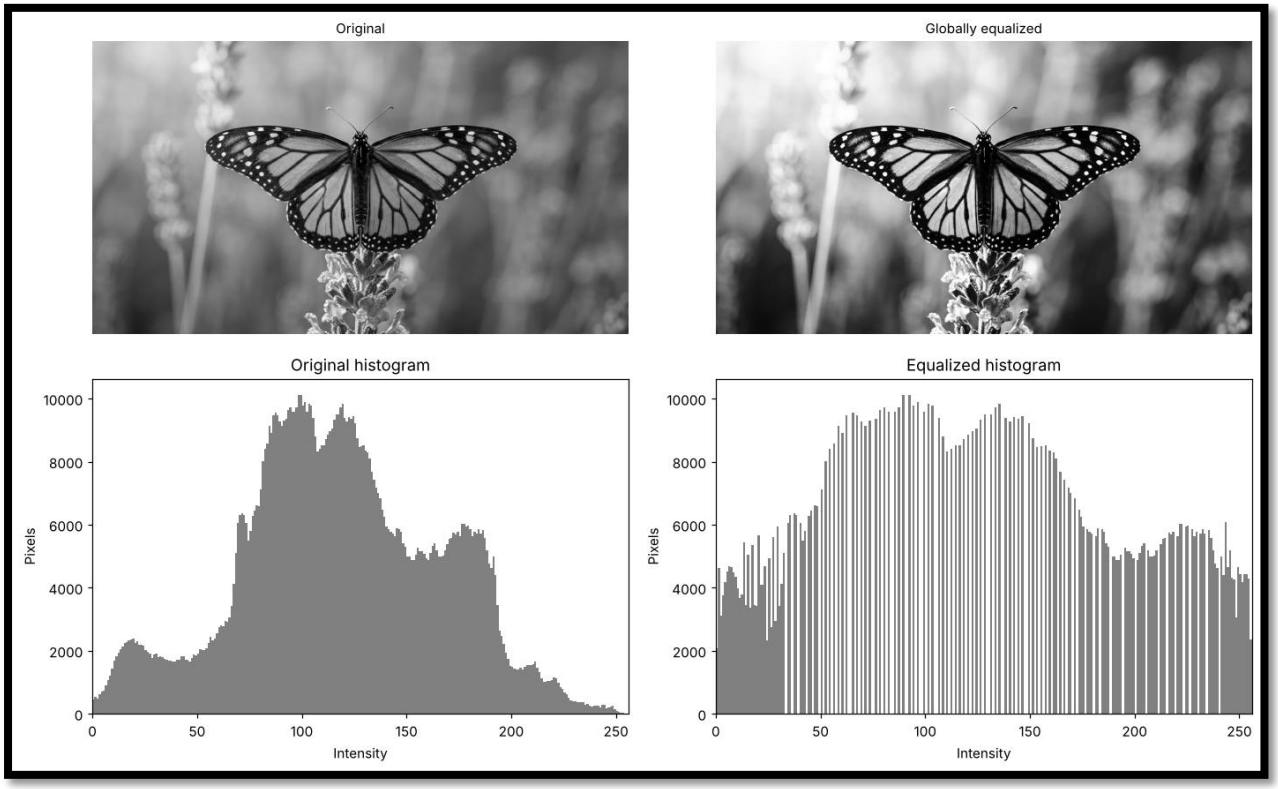


Figure 7 – Original and equalized images with their histograms.

Equalization spreads the mid-tone range across 0–255 (std 47.6 → 73.6), so wing veins, the flower and the background texture become more visible and the overall contrast rises. Noise also becomes stronger: the Laplacian (high-frequency) standard deviation increases from 23.3 to 34.5, which means that grain, JPEG texture and blur boundaries in the flat background are amplified. The histogram is wider but has gaps (comb shape) because only 256 discrete levels can be remapped.

Dry run 7 – pixels [50, 50, 100, 150]

intensity	freq	cumulative	CDF	255·CDF	map 1	255(CDF−CDFmin)/(1−CDFmin)	map 2
50	2	2	0.5	127.5	128	0	0
100	1	3	0.75	191.25	191	127.5	128
150	1	4	1	255	255	255	255

Table 8 – Equalization dry run (CDFmin = 0.5; round half up)

Output list, method 1 (255 × CDF): [128, 128, 191, 255]

Output list, method 2 (CDF-min formula): [0, 0, 128, 255]

OpenCV cv2.equalizeHist: [0, 0, 128, 255] – identical to method 2.

Method 1 maps the darkest value (50) to 128, so black is never reached; method 2 subtracts CDFmin so the lowest level becomes 0 and the highest 255, which is what OpenCV does.

8. CLAHE (clipLimit = 2.0, tileGridSize = (8, 8))

```
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
cl = clahe.apply(img)

fig, ax = plt.subplots(2, 3, figsize=(16, 8))
show(ax[0,0], img, "Original")
show(ax[0,1], eq,  "Global equalization")
show(ax[0,2], cl,  "CLAHE (clip 2.0, 8x8 tiles)")
for k, (im, t) in enumerate([(img,"Original"), (eq,"Global equalization"), (cl,"CLAHE")]):
    ax[1,k].hist(im.ravel(), bins=256, range=(0,256), color="gray"); ax[1,k].set_title(t+" histogram")
    ax[1,k].set_xlim(0,256)
plt.tight_layout(); save_fig("08_clahe"); plt.show()

# zoom into a dark background crop and a wing crop for local-detail comparison
crop = (slice(450, 750), slice(0, 400))
fig, ax = plt.subplots(1, 3, figsize=(14, 4))
show(ax[0], img[crop], "Crop: original")
show(ax[1], eq[crop],  "Crop: global equalization")
show(ax[2], cl[crop],  "Crop: CLAHE")
plt.tight_layout(); save_fig("08_clahe_crop"); plt.show()

print("Std of intensities  orig / global / CLAHE:", round(img.std(),2), round(eq.std(),2), round(cl.std(),2))
print("Laplacian std       orig / global / CLAHE:", round(lap(img),2), round(lap(eq),2), round(lap(cl),2))
```

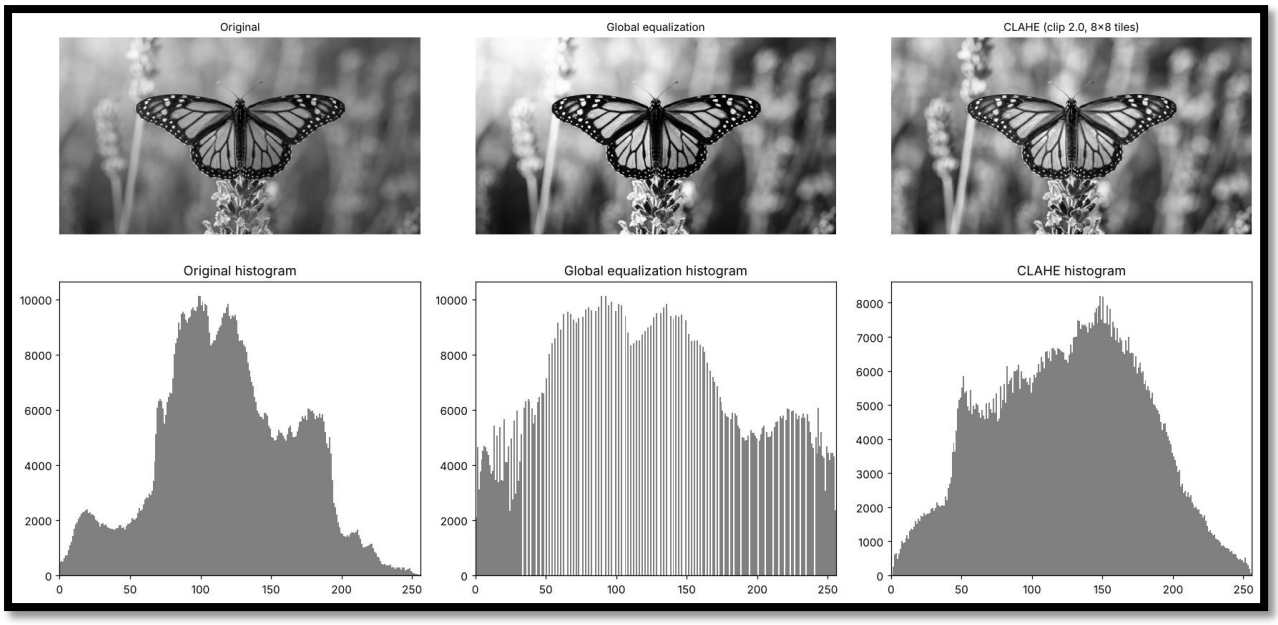


Figure 8 – Original, global equalization and CLAHE with histograms.

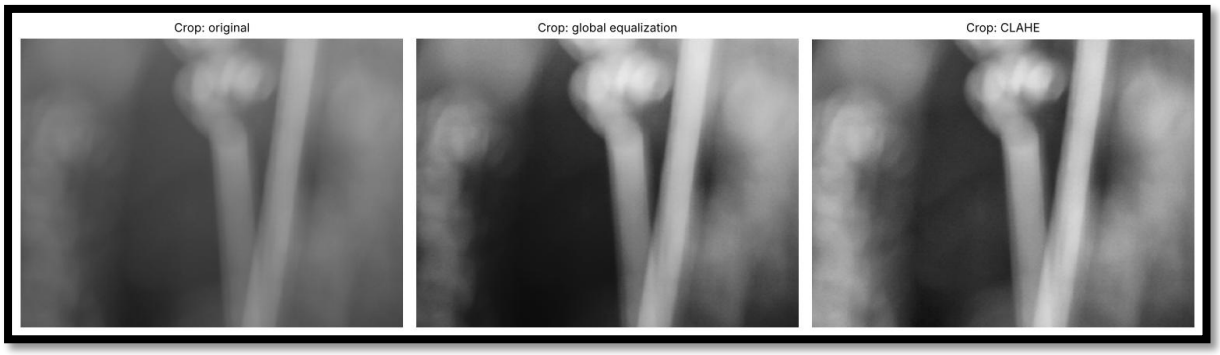


Figure 9 – Crop of the bottom-left (dark, blurred) region for all three versions.

Comparison: global equalization uses a single mapping for the entire image (std 73.6, Laplacian 34.5), so it over-stretches the large smooth background and brightness differences between regions remain. CLAHE builds a mapping per tile and limits the slope of each mapping, so contrast is improved locally (std 53.1) while the noise gain is smaller (Laplacian 30.6) and the result looks more natural, which is useful when lighting is uneven. tileGridSize = (8, 8) means the image is split into 8 × 8 = 64 tiles (8 columns × 8 rows); it is not a tile size of 8 × 8 pixels. Here every tile is about 176 × 96 pixels.

Dry run 8 – histogram [6, 2, 0, 0], absolute cap 4

bin	original	clipped (cap 4)	+ excess/4	CDF old	CDF new	255 × CDF new
0	6	4	4.5	0.75	0.5625	143.4375
1	2	2	2.5	1	0.875	223.125
2	0	0	0.5	1	0.9375	239.0625
3	0	0	0.5	1	1	255

Table 9 – CLAHE clipping and redistribution

Excess = (6 − 4) = 2; shared equally over four bins → +0.5 each, giving [4.5, 2.5, 0.5, 0.5] (total still 8). The new CDF is [0.5625, 0.875, 0.9375, 1.0], so the bin-0 mapping changes from 255 × 0.75 = 191 to 143.4: the steep rise caused by the tall bin is reduced. Neighbouring tiles need interpolation because each tile

has its own mapping; without bilinear blending between tile mappings the output would show visible block edges at tile borders. This simplified model differs from OpenCV's integer redistribution, and the absolute cap is not the clipLimit parameter.

9. Fixed binary thresholding (T = 127)

```
T = 127
_, bw127 = cv2.threshold(img, T, 255, cv2.THRESH_BINARY)

fig, ax = plt.subplots(1, 2, figsize=(12, 4))
show(ax[0], img,   "Original")
show(ax[1], bw127, "Fixed threshold T = 127")
plt.tight_layout(); save_fig("09_fixed_threshold"); plt.show()

print("Unique values in the binary image:", np.unique(bw127))
print("White pixels: %d (%.2f %%)" % (np.sum(bw127==255), 100*np.mean(bw127==255)))
```



Figure 10 – Original and fixed-threshold binary image.

Unique values in the binary image: [0, 255]. White pixels: 433,990 (40.13 %). The bright flower heads and wing spots become white, dark wing borders and the shaded background become black.

Dry run 9 – pixels [0, 80, 127, 128, 180, 255]

r	T = 127 (r > 127)	T = 180 (r > 180)
0	0	0
80	0	0
127	0	0
128	255	0
180	255	0
255	255	255

Table 10 – Fixed thresholding dry run

White-pixel counts: T = 127 → 3 (128, 180, 255); T = 180 → 1 (only 255). Because the rule is r > T, the pixels equal to T (127 and 180) stay black. A higher threshold keeps fewer, brighter pixels. OpenCV gives the same results.

10. Otsu thresholding

```
t_otsu, bw_otsu = cv2.threshold(img, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
print("Otsu threshold returned by OpenCV:", t_otsu)

diff = bw127 != bw_otsu
fig, ax = plt.subplots(1, 3, figsize=(16, 4))
show(ax[0], bw127,    "Fixed T = 127")
show(ax[1], bw_otsu, f"Otsu T = {t_otsu:.0f}")
show(ax[2], (diff*255).astype(np.uint8), "Pixels where the masks differ")
plt.tight_layout(); save_fig("10_otsu"); plt.show()

print("White pixels fixed / Otsu : %.2f %% / %.2f %%" % (100*np.mean(bw127==255), 100*np.mean(bw_otsu==255)))
print("Pixels that differ        : %d (%.2f %%)" % (diff.sum(), 100*diff.mean()))
```

```
Otsu threshold returned by OpenCV: 122.0
```

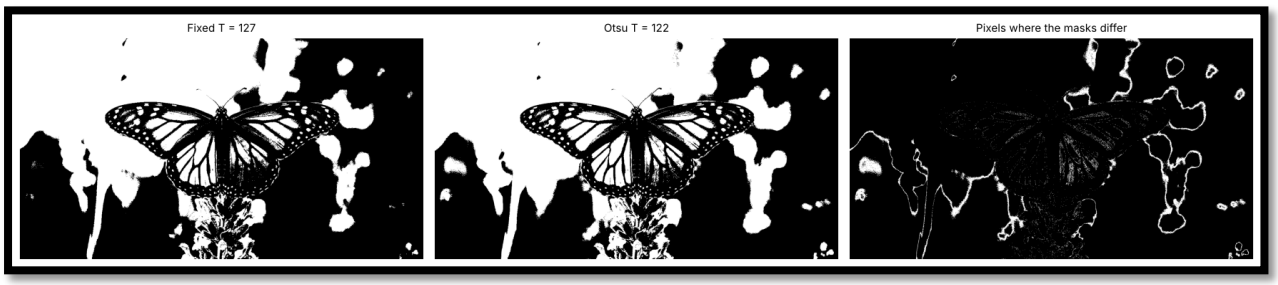


Figure 11 – Fixed T = 127, Otsu T = 122 and the pixels where the two masks differ.

Returned threshold: 122.0. The Otsu mask has 44.32 % white pixels against 40.13 % for T = 127; 45,260 pixels (4.19 %) differ. Otsu chose a slightly lower threshold because it picks the value that maximises the between-class variance of this image's own histogram, rather than a fixed guess, so more mid-grey pixels (the lighter wing areas) become white.

Dry run 10 – pixels [0, 1, 4, 5]

T	class 0 (r ≤ T)	class 1 (r > T)	w0	w1	μ0	μ1	$w0 \cdot w1 \cdot (\mu0-\mu1)^2$
0	[0]	[1, 4, 5]	0.25	0.75	0	3.3333	2.0833
1	[0, 1]	[4, 5]	0.5	0.5	0.5	4.5	4
2	[0, 1]	[4, 5]	0.5	0.5	0.5	4.5	4
3	[0, 1]	[4, 5]	0.5	0.5	0.5	4.5	4
4	[0, 1, 4]	[5]	0.75	0.25	1.6667	5	2.0833

Table 11 – Otsu dry run

The maximum between-class variance is 4.0 (T = 1, 2 and 3 tie). Choosing the smallest tied T gives T = 1. Predicted mask for r > 1: [0, 0, 255, 255]. OpenCV returns threshold 1.0 and mask [0, 0, 255, 255], so the prediction matches.

11. Adaptive Gaussian thresholding (blockSize = 11, C = 2)

```
adapt = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                              cv2.THRESH_BINARY, 11, 2)

fig, ax = plt.subplots(2, 2, figsize=(13, 8))
show(ax[0,0], img,      "Original")
show(ax[0,1], bw127,    "Fixed T = 127")
show(ax[1,0], bw_otsu,  f"Otsu T = {t_otsu:.0f}")
show(ax[1,1], adapt,    "Adaptive Gaussian (11, C=2)")
plt.tight_layout(); save_fig("11_adaptive"); plt.show()

for name, m in [("fixed", bw127), ("otsu", bw_otsu), ("adaptive", adapt)]:
    print(f"{name:9s} white pixels: {100*np.mean(m==255):.2f} %")
```

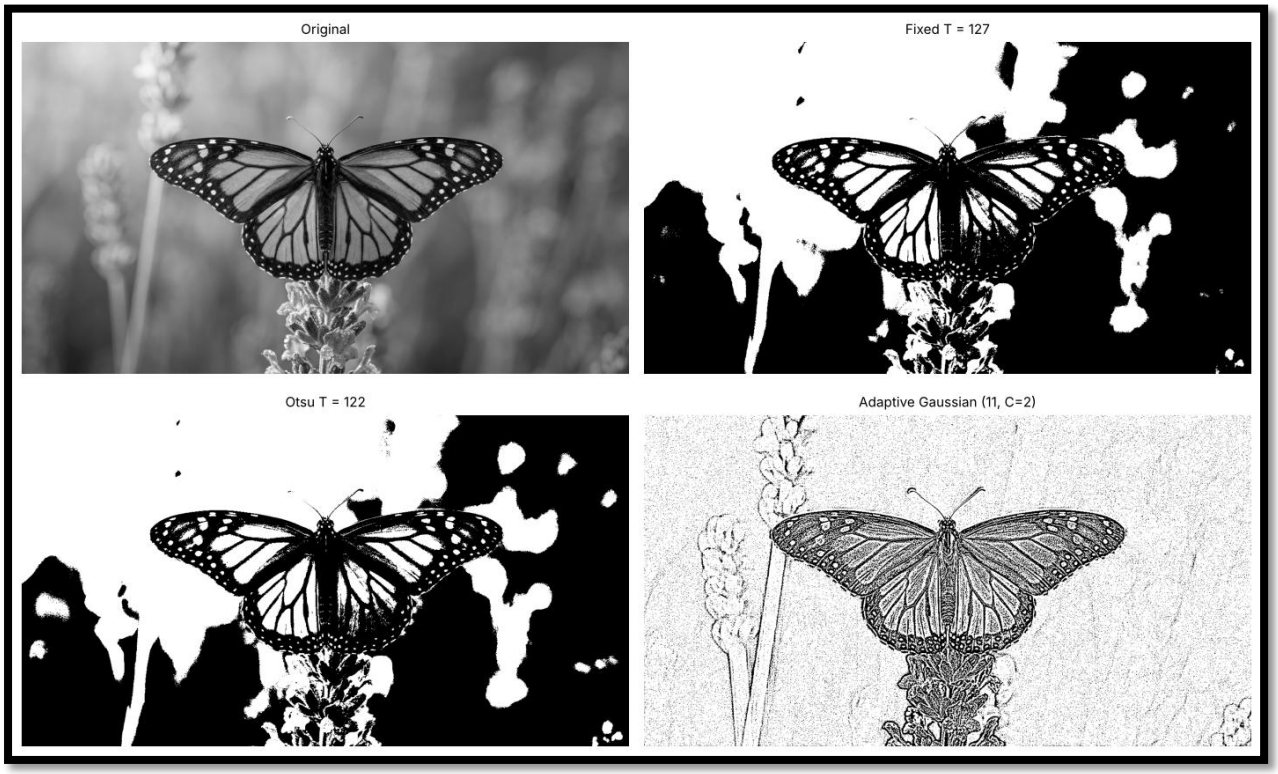


Figure 12 – Original, fixed, Otsu and adaptive Gaussian thresholding.

Method	White pixels
Fixed T = 127	40.13 %
Otsu T = 122	44.32 %
Adaptive Gaussian (11, C = 2)	84.42 %

Table 12 – White-pixel share of the three thresholding methods

Comparison: fixed and Otsu thresholds use one global cut-off, so they separate large bright regions (flowers) from dark regions and ignore local variations. The adaptive threshold uses the weighted mean of each 11 × 11 neighbourhood minus C, so it outlines wing veins, spots, antennae and flower edges even where the absolute brightness is the same, but flat regions (e.g. blurred background) become white with scattered black noise (84 % white). Local thresholds help when illumination is uneven, e.g. shadows on scanned text or a gradient background.

Dry run 11 – 3 × 3 neighbourhood

Neighbourhood N = [[100, 100, 100], [100, 120, 140], [140, 140, 140]]; weights W = [[1, 2, 1], [2, 4, 2], [1, 2, 1]] / 16.

	col 1	col 2	col 3
row 1	6.25	12.5	6.25
row 2	12.5	30	17.5
row 3	8.75	17.5	8.75

Table 13 – Weighted products N × W (sum = 120)

Weighted mean = 1920 / 16 = 120; T = mean − C = 120 − 2 = 118. Centre pixel 120 > 118 → output 255. A pixel equal to T (118) is not greater than T → output 0. (The dry run uses 3 × 3; the image task uses 11 × 11.)

Explanations required by the submission instructions

Clipping: forcing out-of-range results to 0 or 255 (np.clip). It prevents wrap-around but is irreversible – every value beyond the limit is lost (saturation).

Overflow: uint8 arithmetic is modulo 256, so 250 + 40 = 34. It is avoided by converting to int16/float first, clipping, and only then casting back to uint8.

Brightness versus contrast: brightness adds a constant β and shifts the whole histogram; contrast multiplies by α and stretches (α > 1) or compresses (α < 1) the differences. Centring at 128 changes contrast without shifting the mid-grey level, whereas αr also changes brightness.

Gamma convention: s = 255(r/255)^γ. γ < 1 brightens dark regions, γ > 1 darkens; 0 and 255 never change.

Global versus local enhancement/thresholding: global methods (equalizeHist, fixed, Otsu) are simple and work when the lighting is uniform; local methods (CLAHE, adaptive thresholding) adapt to each neighbourhood and are better with uneven lighting or shadows but can amplify noise.

Conclusion

All eleven operations were implemented and every dry run was calculated by hand and verified in Python (brightness, contrast, negative, gamma, histogram, equalization – matching OpenCV exactly, CLAHE model, fixed thresholds, Otsu – matching OpenCV exactly, and adaptive Gaussian). The experiments show that safe arithmetic (wider types plus clipping) is essential, that centred contrast preserves overall brightness, and that local methods are more suitable than global ones when illumination varies.